

Automatic redirects and history

Notice that if you try to get Google's homepage through `www.google.com`, Requests gives you the homepage. But `www.google.com` doesn't really serve you a page but simply redirects you to Google's country-specific homepage for the country you're accessing the homepage from. What's going on here?

Well, Requests automatically follows all HTTP redirects for all verbs except HEAD. That means GET, PUT, POST, DELETE and PATCH are covered. And just because Requests does it automatically for these verbs doesn't mean you can't turn it off, or that you can't turn it on for HEAD.

Let's see what's going on here:

```
>>> r = requests.get("https://www.google.com")
>>> r.url
'https://www.google.co.in/'
>>> r.status_code
200
```

So we queried for `www.google.com`, but got back `www.google.co.in`, with a 200 OK status code. What happened to the Request to `www.google.com`?

```
>>> r.history
[<Response [302]>]
```

That's the *history* of all the requests that had to be completed to get us to the 200 OK response. There's only one of them here for this request, but if Requests had to go through more than one redirection, this list would be sorted from the oldest to the most recent request.

Let's try something else:

```
>>> r.history[0].url
'https://www.google.com/'
>>> r.history[0].status_code
302
```

So the history list is actually a list of full-fledged Response objects. You could inspect all of them and grab whatever data you wanted - status codes, headers, URLs—from them.

What if you didn't want the redirections to be followed? You could do the following:

```
>>> r = requests.get("https://www.google.com", allow_
redirects=False)
>>> r.url
'https://www.google.com/'
>>> r.status_code
302
```

And if you wanted your HEAD request to follow all redirects, you'd do as follows:

```
>>> r = requests.head("https://www.google.com", allow_
redirects=True)
>>> r.url
'https://www.google.co.in/'
>>> r.status_code
200
```

By default, Requests will resolve 30 redirects before giving up with a *TooManyRedirects* error. You can change that, but you'd have to create a Session first, which I'll go into later. Let's do some authentication now.

Authentication

HTTP was built to have protected resources, and REST relies heavily on authentication. Requests makes HTTP authentication almost too easy.

Let's start with HTTP Basic Authentication, which apart from being easy, since it's done over SSL, is pretty secure and pretty common. So common, in fact, that Requests provides a handy shorthand to do it:

```
>>> requests.get("https://api.github.com/user", auth=("user",
"pass"))
<Response [200]>
```

The Requests authentication system is modular, and authentication modules can be plugged-in per request or per session. The long way to do HTTP Basic Authentication is as follows:

```
>>> from requests.auth import HTTPBasicAuth
>>> requests.get("https://api.github.com/user",
auth=HTTPBasicAuth("user", "pass"))
<Response [200]>
```

HTTPBasicAuth is an authentication plug-in and it does HTTP Basic Authentication. There are plenty more of these authentication plug-ins, and some of them come with Requests itself. For example, to do HTTP Digest Authentication, you'd use the *HTTPDigestAuth* plug-in, as shown below:

```
>>> from requests.auth import HTTPDigestAuth
>>> url = "http://example.com/my_digest_endpoint"
>>> requests.get(url, auth=HTTPDigestAuth("user", "pass"))
<Response [200]>
```

There are third party libraries that allow other authentication methods. You can do Kerberos authentication by installing the *requests-kerberos* package (from PyPI), OAuth 1 and OAuth 2 by installing the *requests-oauthlib* package, NTLM by installing the *requests_ntlm* package, and there's even a package for AWS authentication (the PyPI package is called *requests-aws*).